

# Sorting Algorithms

Bubble, Selection, Insertion, Merge & Quick Sort

*Concept • Steps • Time Complexity • Comparison*

# How Sorting Algorithms Are Classified

Before comparing individual algorithms, it helps to know the properties used to describe them.



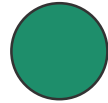
## Comparison-Based

Sorts by comparing pairs of elements (e.g. Bubble, Merge, Quick).



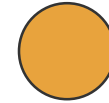
## Non-Comparison

Sorts using element properties instead of comparisons (e.g. Counting Sort).



## Stable

Equal elements keep their original relative order after sorting.



## In-Place

Sorts using little to no extra memory beyond the input array.

*This deck covers five widely-taught sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort.*

# Bubble Sort

Repeatedly compares adjacent elements and swaps them if they are in the wrong order. Larger elements gradually "bubble up" to the end of the array.

- 1 Compare each pair of adjacent elements
- 2 Swap them if the left one is larger
- 3 Repeat the full pass across the array
- 4 Stop when a full pass makes no swaps

Time Complexity

$O(n^2)$

*worst & average case*

Stable?

Yes

In-Place?

Yes

**Example:**  $[5, 1, 4, 2] \rightarrow [1, 4, 2, 5] \rightarrow [1, 2, 4, 5]$  after repeated passes.

# Selection Sort

Divides the array into a sorted and an unsorted part. On each pass, it finds the smallest element in the unsorted part and moves it to the end of the sorted part.

- 1 Find the minimum element in the unsorted portion
- 2 Swap it with the first unsorted position
- 3 Move the sorted/unsorted boundary forward
- 4 Repeat until the whole array is sorted

Time Complexity

$O(n^2)$

*worst, average & best case*

Stable?

No

In-Place?

Yes

**Example:** [29, 10, 14, 37] → smallest is 10 → [10, 29, 14, 37] → continue with the rest.

# Insertion Sort

Builds the sorted array one element at a time, taking each new element and inserting it into its correct position among the already-sorted elements.

- 1 Start from the second element
- 2 Compare it with elements before it
- 3 Shift larger elements one step right
- 4 Insert the current element in its correct slot

**Time Complexity**

**$O(n^2)$  /  $O(n)$**

*worst case / best case — fast on nearly-sorted data*

**Stable?**

**Yes**

**In-Place?**

**Yes**

**Example:** [12, 11, 13, 5] → insert 11 before 12 → [11, 12, 13, 5] → continue inserting 5.

# Merge Sort

A divide-and-conquer algorithm. It splits the array in half recursively until single elements remain, then merges the halves back in sorted order.

- 1 Divide the array into two halves
- 2 Recursively sort each half
- 3 Merge the two sorted halves together
- 4 Repeat merging until one sorted array remains

Time Complexity

**$O(n \log n)$**

*worst, average & best case*

Stable?

**Yes**

In-Place?

**No**

**Example:** [38, 27, 43, 3] → split into [38, 27] and [43, 3] → sort and merge → [3, 27, 38, 43].

# Quick Sort

Also divide-and-conquer, but it picks a pivot element and partitions the array so smaller elements go left and larger elements go right, then sorts each side.

- 1 Choose a pivot element from the array
- 2 Partition: smaller elements left, larger right
- 3 Recursively sort the left partition
- 4 Recursively sort the right partition

Time Complexity

$O(n^2)$

*worst case;  $O(n \log n)$  on average*

Stable?

No

In-Place?

Yes

**Example:** [10, 80, 30, 90, 40] with pivot 40 → [10, 30] | 40 | [80, 90, 40] → sort each side.

## All Five, Side by Side

| Algorithm      | Best          | Worst         | Stable | In-Place |
|----------------|---------------|---------------|--------|----------|
| Bubble Sort    | $O(n)$        | $O(n^2)$      | Yes    | Yes      |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | No     | Yes      |
| Insertion Sort | $O(n)$        | $O(n^2)$      | Yes    | Yes      |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | Yes    | No       |
| Quick Sort     | $O(n \log n)$ | $O(n^2)$      | No     | Yes      |

**Rule of thumb:** Use Merge or Quick Sort for large datasets. Use Insertion Sort for small or nearly-sorted data. Bubble and Selection Sort are mainly used for teaching.